CS 0007: Introduction to Java Lecture 7

Nathan Ong
University of Pittsburgh
September 22, 2016

FUNCTIONS

Functions

- A function is similar to its mathematical counterpart
- $f(x) = x^2$, plug in 3, get 9
- Contains several more parts

Java Functions

- Takes in zero or more parameters, processes them in the function body, and returns a result
- Imagine going to BestBuy™ and telling them you want your computer fixed. You are telling them to run a fixing function, with your computer being a parameter. What you get back is your fixed computer.

You Already Have the Power!

- You already know how to call functions!

Static:

ClassName.functionName(<parameters>);

Non-static:

objectName.functionName(<parameters>);

- How do I make my own?

Function Components

Function Header

1. Visibility type (`public`/`protected`/`private`)
2. `static` (For now, required)
3. Return Type
4. functionName
5. Parentheses “()”
   - Parameters
   a) Type1 parameterName1
   b) Type2 parameterName2
   c) …

Function Body

6. Curly Brackets/Braces “{}”
   - `return` a value

A Simple Unnecessary Function

- I want this function to take two `doubles` and return their sum.
- Let us go through the list and see what needs to be incorporated for the function.

Function Components

1. We'll just use `public` for now.
2. `static` (For now, required)
3. What is the Return Type?
4. What is an appropriate functionName?
5. Parentheses "()"
   - What are the parameters?
6. Curly Brackets/Braces "{}"
   - What do we do in the Function Body?
   - What do we `return`?

Return Type

- What kind of thing are we going to give back to the function caller?
- “I want this function to take two `double`s and return their sum.”
- The sum of two `double`s better be a `double`.

Function Name

- The function name should easily describe what the function does.
- “I want this function to take two `doubles` and return their sum.”
- “sum”? Probably not enough detail, since there are many types that can be summed.
- sumDoubles

Parameters

- The required input to the function.
- The data that the function needs in order to properly execute its duties.
- “I want this function to take two `double`s and return their sum.”
- Two `double`s.
- Names?

Function Header

```
//sums two doubles together
public static double sumDoubles
   (double addend1, double addend2)
```

Function Body

```
...(Function Header)
{
   ???
}
```

Function Body

...(Function Header)

```
{
  double sum = addend1 + addend2;
  return sum;
}//end method(double,double)
```

Function Body

...(Function Header)

```
{
  return addend1 + addend2;
}//end method(double,double)
```

Function Body

```
public class Functions
{
  …(Function)

  public static void main(String[] args)
  {
  double sum = sumDoubles(2.5,3.9);
  System.out.println(sum);
  }//end method main
}//End class Functions
```

```java
import java.util.Scanner;

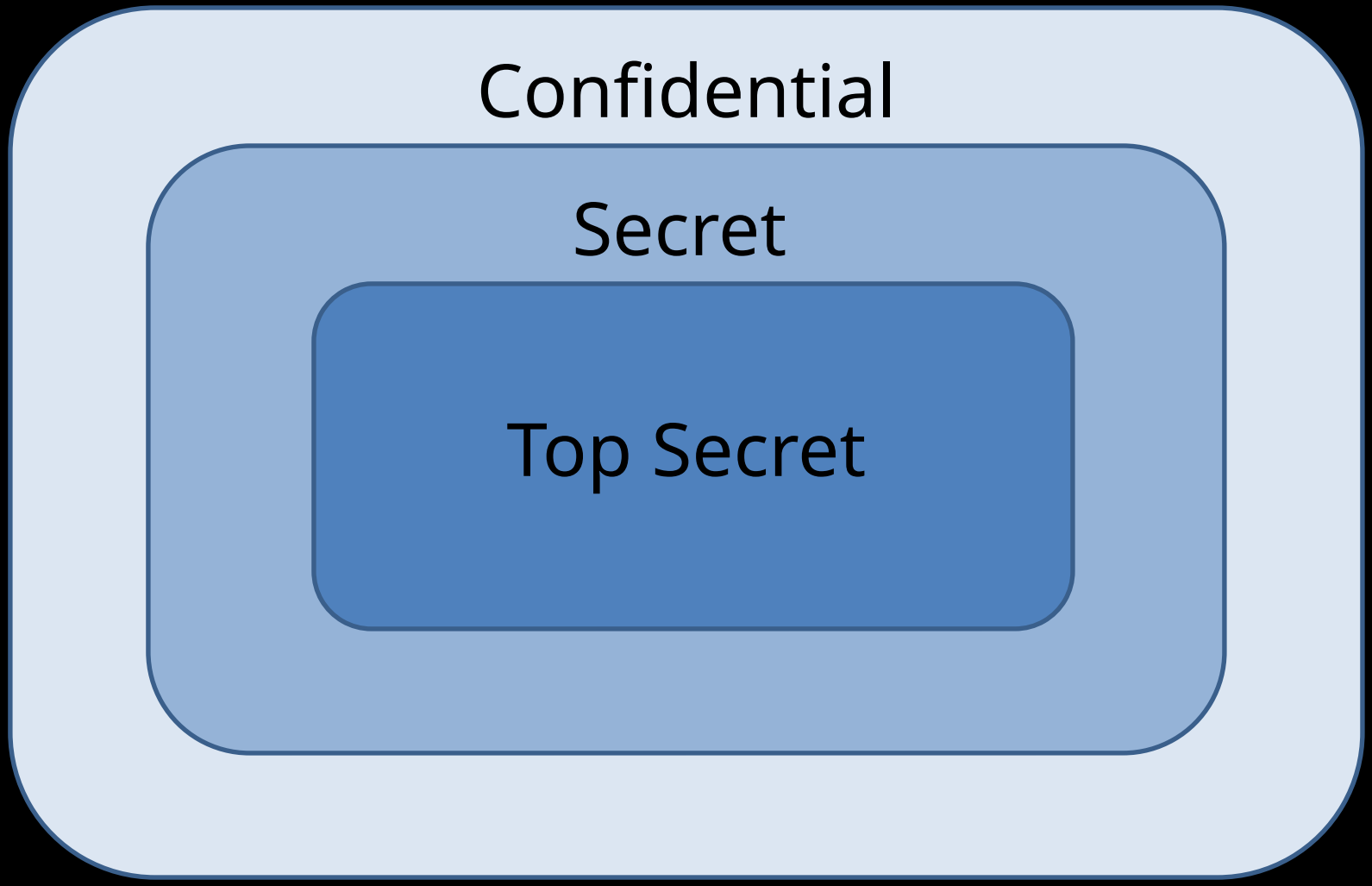
public class AddingMachine
{
   ...(Function)
   public static void main(String[] args)
   {
          Scanner scan = new Scanner(System.in);
          System.out.println("Please enter a number:");
          double firstNum = scan.nextDouble();
          System.out.println("Please enter a second
   number:");
          double secondNum = scan.nextDouble();

          double sum = sumDoubles(firstNum,secondNum);
          System.out.println("The sum of " + firstNum +
                        " and " + secondNum + " equals " +
                        sum);
   }//end method main
}//End class AddingMachine
```

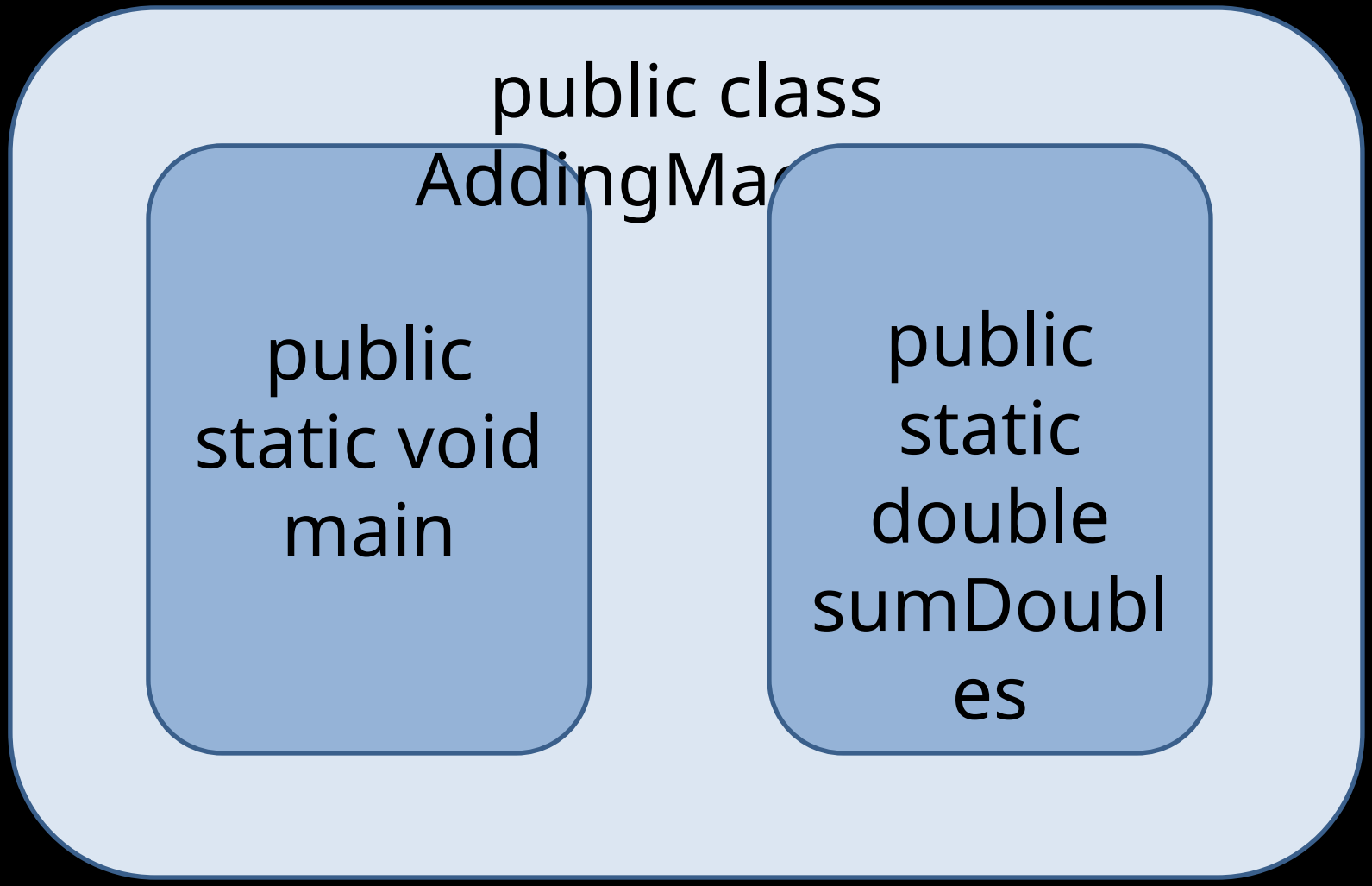
Scope

- Why did we need to submit **`firstNum`** and **`secondNum`** to **`sumDoubles`**?
- Why can't **`sumDoubles`** just use the already created variables?
- This relates to scope.

Scope



Scope


public class AddingMa
public static void main
public static double sumDoubles

Passing-in Parameters

- The act of providing parameters in a function call is called Passing-in.
- Java has two kinds of passing-in.

Pass-by-Value

- Passing-in a copy of the value of the variable.
- Any change made to the variable is not reflected when the function returns.
- All primitive type parameters are pass-by-value.

Pass-by-Reference

- Passing-in the actual variable.
- Any change made to the variable IS reflected when the function returns, UNLESS the parameter name is REASSIGNED.
- All non-primitive type parameters are pass-by-reference.

Another Simple Function

- I want a function that prints a nice British greeting, complete with the person's full name.

Source: http://www.bbcamerica.com/anglophenia/2011/07/five-slang-ways-to-say-hello/

Function Components

1. `public`
2. `static`
3. What is the Return Type?
4. What is an appropriate functionName?
5. Parentheses “()”
   - What are the parameters?
6. Curly Brackets/Braces “{}”
   - What do we do in the Function Body?
   - What do we `return`?

Return Type

- Does this function even need to return any data or information?
- No, so we should use `void` as our return type.

Function Header

```
public static void britishGreeting
      (String fullName)
```

Function Body

...(Function Header)

```
{
  System.out.println("Wotcha " +
            fullName +
            ".  Fancy a cuppa?");
}//end method(String)
//Note no return statement
```